Experiment No: 4

Title :

Use function types, scope, and closures; apply try-catch; build a palindrome checker.

Software/Tools Required :

- Any text/code editor (e.g., VS Code)
- Web browser (e.g., Chrome, Edge, Firefox) to render the HTML page
- Basic knowledge of HTML5 and modern JavaScript (ES6+)

Theory :

JavaScript Functions

Functions are reusable blocks of code designed to perform a specific task. Instead of writing the same set of instructions again and again, functions let a developer define a piece of logic once and reuse it wherever it's needed, which makes programs shorter, cleaner, and easier to maintain. JavaScript supports different types of functions:

- **Function Declaration:** A named function that can be called before its definition due to hoisting. Hoisting means the JavaScript engine moves function declarations to the top of their scope in memory before the code actually runs, so the function is fully available even if it is called earlier in the file than where it is written.
- **Function Expression:** A function assigned to a variable, executed only after initialization. Since the function is stored as a value inside a variable, it does not get hoisted the same way as a declaration trying to call it before the assignment line runs will cause an error.
- **Arrow Function:** A concise ES6 syntax for writing functions, especially useful for callbacks and short functions. Arrow functions have a shorter syntax than regular functions and, unlike normal functions, they do not have their own this value they simply use the this of the code around them, which makes them very handy inside other functions or methods.

Variable Scope

Scope determines where variables can be accessed. Understanding scope is important because it decides which part of a program can "see" and use a particular variable, and it helps avoid naming conflicts and unexpected bugs.

- **Global Scope:** Variables declared outside functions are accessible throughout the program. While convenient, relying too heavily on global variables can make code harder to debug since any part of the program can change them.
- **Function Scope:** Variables declared using var inside a function are accessible only within that function. This means once the function finishes running, those variables are no longer available outside of it.
- **Block Scope:** Variables declared with let and const are limited to the block ({}) in which they are defined. This includes blocks like if statements, loops, and even standalone curly braces, making these variables more predictable and safer to use than var.

Closures

A closure is created when an inner function retains access to the variables of its outer function even after the outer function has finished execution. This happens because the inner function keeps a memory of the environment in which it was created, not just the values that existed at that moment. Closures are commonly used for data encapsulation, private variables, and maintaining state, since JavaScript does not have built-in private variables the way some other programming languages do closures act as a natural way of achieving that same effect. They are also widely used in situations like counters, timers, and event-based programming, where a function needs to "remember" something between multiple calls.

Exception Handling (try-catch)

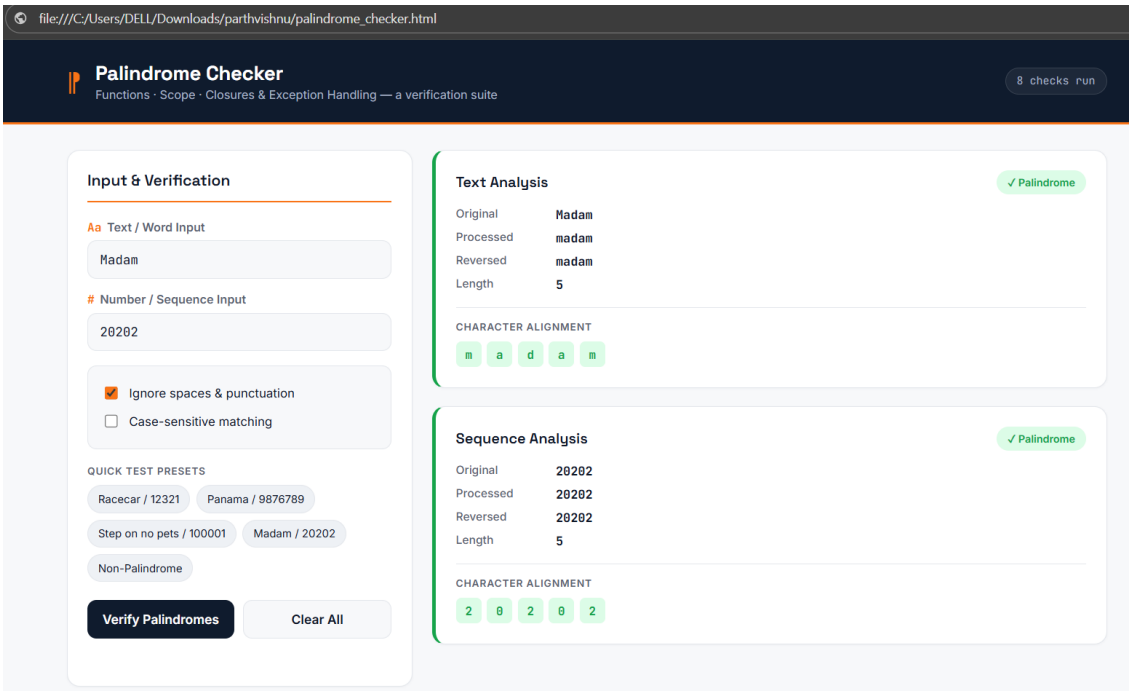
The try-catch statement allows JavaScript programs to handle runtime errors gracefully. The try block contains code that may generate an error, while the catch block executes if an exception occurs, preventing the application from crashing. This is especially useful when working with things that are unpredictable, such as user input, external data, or network requests, since errors in these areas are common and shouldn't stop the entire program. Along with try and catch, JavaScript also allows an optional finally block, which runs no matter what happens whether an error occurred or not making it useful for cleanup tasks such as resetting values or closing processes.

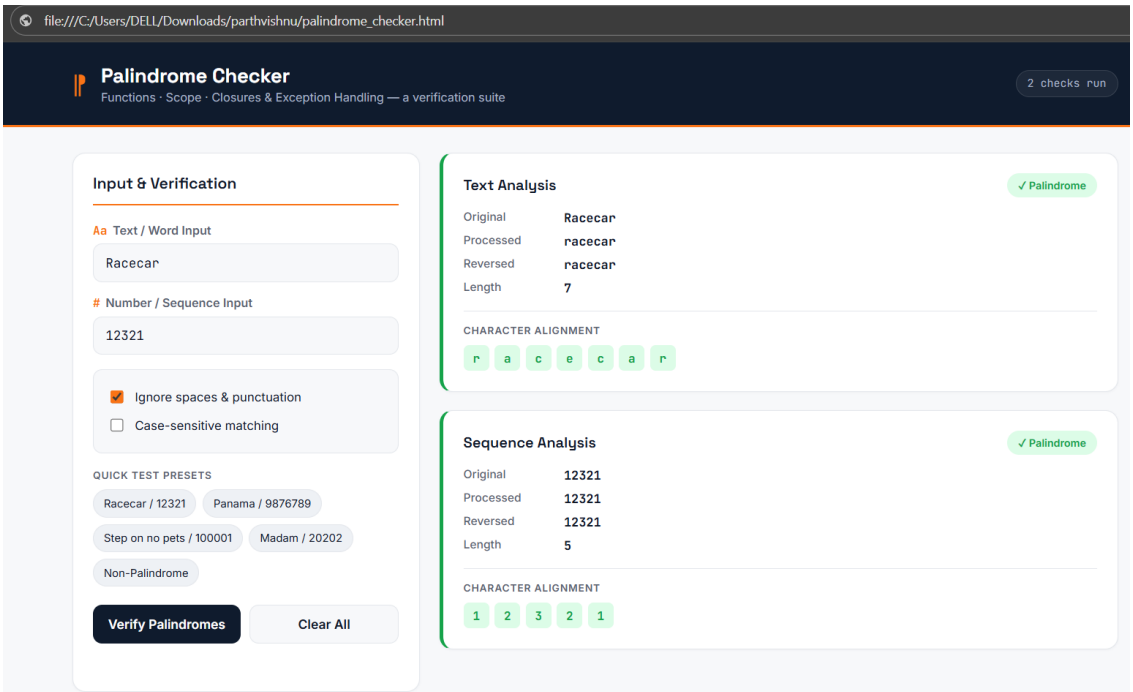
Aim of the CASE STUDY:

This experiment involves creating an interactive **Palindrome Checker**. The application accepts a string from the user and determines whether it is a palindrome. It demonstrates different function types for processing input, uses variable scope effectively, implements closures to preserve internal state, and applies try-catch to handle invalid or unexpected user input.

Name : Satish Purohit PRN : 24070521268

File path - file:///C:/Users/DELL/Downloads/satishpurohit/palindrome_checker.html





Output :

- The Palindrome Checker was successfully implemented and tested in JavaScript.
- Different function types (declaration, expression, arrow) correctly processed the user input to check for palindromes.
- Variable scope was properly maintained, separating global and local data during execution.
- Closures successfully preserved internal state, such as tracking previous checks.
- The try-catch block handled invalid inputs (empty or non-string values) without crashing the program.
- The program accurately identified palindromes and non-palindromes, confirming correct functionality.

Result / Conclusion :

The webpage was successfully created and executed. The output verifies the correct implementation of JavaScript function types, variable scope, and closures. The experiment also demonstrates effective exception handling using try-catch and successfully validates user input to determine whether the entered string is a palindrome, showcasing the practical application of modern JavaScript programming concepts.